

Day 24: Memory Management Advanced

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

Placement New

Placement new constructs an object in **pre-allocated memory** — no heap allocation:

```
// Pre-allocate aligned storage
alignas(Sensor) uint8_t storage[sizeof(Sensor)];

// Construct the object in that storage (no heap)
Sensor *s = new (&storage) Sensor(config);

// When done, call destructor manually (placement new doesn't auto-destruct)
s->~Sensor();
```

Used in object pools, static arenas, and DMA buffers where you need controlled object lifetime.

Memory Pools (K_HEAP_DEFINE)

Zephyr provides bounded heaps via `K_HEAP_DEFINE` :

```
K_HEAP_DEFINE(app_heap, 2048); // 2KB bounded heap

void *buf = k_heap_alloc(&app_heap, 64, K_NO_WAIT);
// ... use buf ...
k_heap_free(&app_heap, buf);
```

Unlike the system heap, this heap is bounded — you know exactly how much memory it can consume.

Custom Allocators Concept

A **custom allocator** replaces `new` / `delete` with your own allocation strategy:

```
void *operator new(size_t size) {
    return k_heap_alloc(&app_heap, size, K_NO_WAIT);
}

void operator delete(void *ptr) noexcept {
    k_heap_free(&app_heap, ptr);
}
```

This lets you use `new` with a bounded heap instead of the system heap.

Smart Pointer Trade-offs in Embedded

`std::unique_ptr`: zero-overhead RAI ownership — compiles to identical code as a raw pointer:

```
auto ptr = std::unique_ptr<uint8_t>(
    static_cast<uint8_t *>(k_heap_alloc(&heap, 64, K_NO_WAIT)),
    [](uint8_t *p) { k_heap_free(&heap, p); }
); // Freed automatically on scope exit
```

`std::shared_ptr`: **avoid in embedded** — uses atomic reference counting (expensive on Cortex-M), and adds 16 bytes of overhead per `shared_ptr`.

unique_ptr for RAI

Use `unique_ptr` when you need heap allocation but want guaranteed cleanup:

```
std::unique_ptr<Bme280Driver> sensor = std::make_unique<Bme280Driver>(spi_dev);
// sensor is automatically deleted when it goes out of scope
```

With a custom deleter (as shown above), `unique_ptr` works with Zephyr's heap allocator.

Memory Pool Patterns

Three common patterns for embedded memory management:

1. **Static allocation** (preferred): `static uint8_t buf[64];` — deterministic, no fragmentation
2. **Object pool** (`k_mem_slab`): fixed-size blocks, deterministic alloc/free, no fragmentation
3. **Bounded heap** (`K_HEAP_DEFINE`): flexible sizes, but possible fragmentation in bounded range

Avoid the system heap (`malloc` / `new`) unless you have `CONFIG_HEAP_MEM_POOL_SIZE` set and understand the fragmentation implications.

Practice Tasks

1. Implement an object pool for `Message` structs using placement new — track alloc/free with a counter
 2. Write a custom `operator new` that allocates from `K_HEAP_DEFINE` — override `operator delete` too
 3. Use `unique_ptr` with a Zephyr heap deleter — verify the destructor is called on scope exit with `LOG_INF`
 4. Profile heap fragmentation: allocate blocks of varying sizes in random order, measure remaining free space over time
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day24
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)

- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day24/main.cpp](#)